

LAB 1

```
# =====
# Lab 1
# =====

# IMPORT LIBRARIES
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Ask for Image to Upload
uploaded = files.upload()

# Load Image
img = cv2.imread(list(uploaded.keys())[0])

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Display the original image
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')
plt.show()
```

sample photo.jpg
sample photo.jpg(image/jpeg) - 3749885 bytes, last modified: 4/13/2026 - 100% done
 Saving sample photo.jpg to sample photo.jpg

Original Image



```
# 1. Image Statistics
print("IMAGE STATISTICS")
print("Shape of image:", img.shape)
print("Data type:", img.dtype)
print("Minimum pixel value:", img.min())
print("Maximum pixel value:", img.max())
print("Mean pixel value:", img.mean())
```

```
IMAGE STATISTICS
Shape of image: (2482, 3737, 3)
Data type: uint8
Minimum pixel value: 0
Maximum pixel value: 255
Mean pixel value: 188.49507642251038
```

```
# 2 Sampling (Downsampling)

# Reduce resolution by skipping pixels
img_down2 = img_rgb[::2, ::2]
img_down4 = img_rgb[::4, ::4]

plt.figure(figsize=(10,4))
```

```
plt.subplot(1,3,1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(img_down2)
plt.title("Downsample x2")
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(img_down4)
plt.title("Downsample x4")
plt.axis('off')

plt.show()
```

Original



Downsample x2



Downsample x4



```
# 3. Quantization (Gray Level Reduction)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
def quantize(image, levels):
    step = 256 // levels
    return (image // step) * step
```

```
q4 = quantize(gray, 4)
q8 = quantize(gray, 8)
q16 = quantize(gray, 16)
```

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,4,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Gray")
plt.axis('off')
```

```
plt.subplot(1,4,2)
plt.imshow(q4, cmap='gray')
plt.title("4 Levels")
plt.axis('off')
```

```
plt.subplot(1,4,3)
plt.imshow(q8, cmap='gray')
plt.title("8 Levels")
plt.axis('off')
```

```
plt.subplot(1,4,4)
plt.imshow(q16, cmap='gray')
plt.title("16 Levels")
plt.axis('off')
```

```
plt.show()
```

Original Gray



4 Levels



8 Levels



16 Levels



```
# 4. RGB CHANNEL EXTRACTION + CORRELATION
```

```
R = img_rgb[:, :, 0]
```

```

G = img_rgb[:, :, 1]
B = img_rgb[:, :, 2]

plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.imshow(R, cmap='gray')
plt.title("Red Channel")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(G, cmap='gray')
plt.title("Green Channel")
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(B, cmap='gray')
plt.title("Blue Channel")
plt.axis('off')

plt.show()

# Correlation
print("CHANNEL CORRELATION")
print("R-G:", np.corrcoef(R.flatten(), G.flatten())[0,1])
print("R-B:", np.corrcoef(R.flatten(), B.flatten())[0,1])
print("G-B:", np.corrcoef(G.flatten(), B.flatten())[0,1])

```

Red Channel



Green Channel



Blue Channel



CHANNEL CORRELATION
R-G: 0.981027448063671
R-B: 0.9589950729138458
G-B: 0.9945037039680079

5. GEOMETRIC TRANSFORMATIONS

```

h, w = img.shape[:2]

# Rotation
center = (w//2, h//2)
rot_matrix = cv2.getRotationMatrix2D(center, 30, 1)
rotated = cv2.warpAffine(img_rgb, rot_matrix, (w, h))

# Scaling
scaled = cv2.resize(img_rgb, None, fx=0.6, fy=0.6)

# Translation
trans_matrix = np.float32([[1, 0, 50], [0, 1, 30]])
translated = cv2.warpAffine(img_rgb, trans_matrix, (w, h))

plt.figure(figsize=(10,4))

plt.subplot(1,4,1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis('off')

plt.subplot(1,4,2)
plt.imshow(rotated)
plt.title("Rotated 30°")
plt.axis('off')

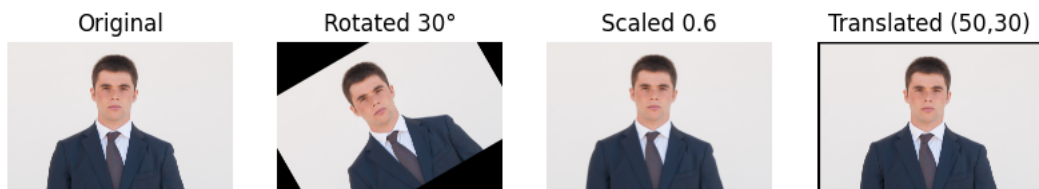
plt.subplot(1,4,3)
plt.imshow(scaled)
plt.title("Scaled 0.6")
plt.axis('off')

plt.subplot(1,4,4)

```

```
plt.imshow(translated)
plt.title("Translated (50,30)")
plt.axis('off')

plt.show()
```



Lab 2

```
# =====
# Lab 2
# =====

# 1. Intensity Transformations - Linear Contrast Stretching
# Expands intensity range to improve contrast
min_val = np.min(gray)
max_val = np.max(gray)

stretch = (gray - min_val) / (max_val - min_val) * 255
stretch = stretch.astype(np.uint8)

# Gamma Correction
gamma = 0.8 # you can try different values
gamma_img = np.array(255 * (gray / 255) ** gamma, dtype='uint8')

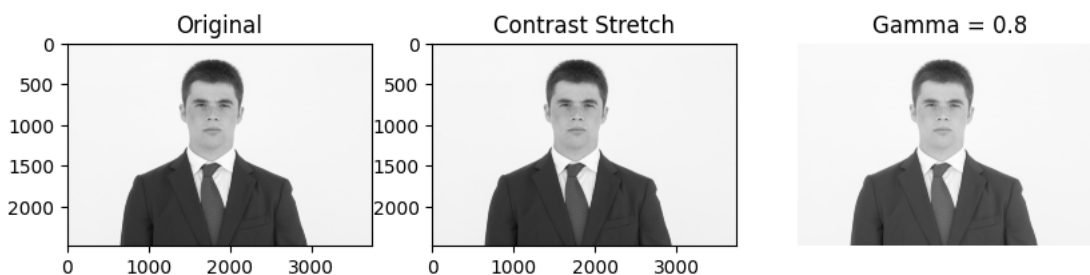
# Display results
plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(stretch, cmap='gray')
plt.title("Contrast Stretch")

plt.subplot(1,3,3)
plt.imshow(gamma_img, cmap='gray')
plt.title(f"Gamma = {gamma}")

plt.axis('off')
plt.show()
```



```
# 2. Histogram Equalization

# Global Histogram Equalization
hist_eq = cv2.equalizeHist(gray)

# CLAHE (Adaptive)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
clahe_img = clahe.apply(gray)

# Display images
```

```
plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(hist_eq, cmap='gray')
plt.title("Global HE")

plt.subplot(1,3,3)
plt.imshow(clahe_img, cmap='gray')
plt.title("CLAHE")

plt.axis('off')
plt.show()

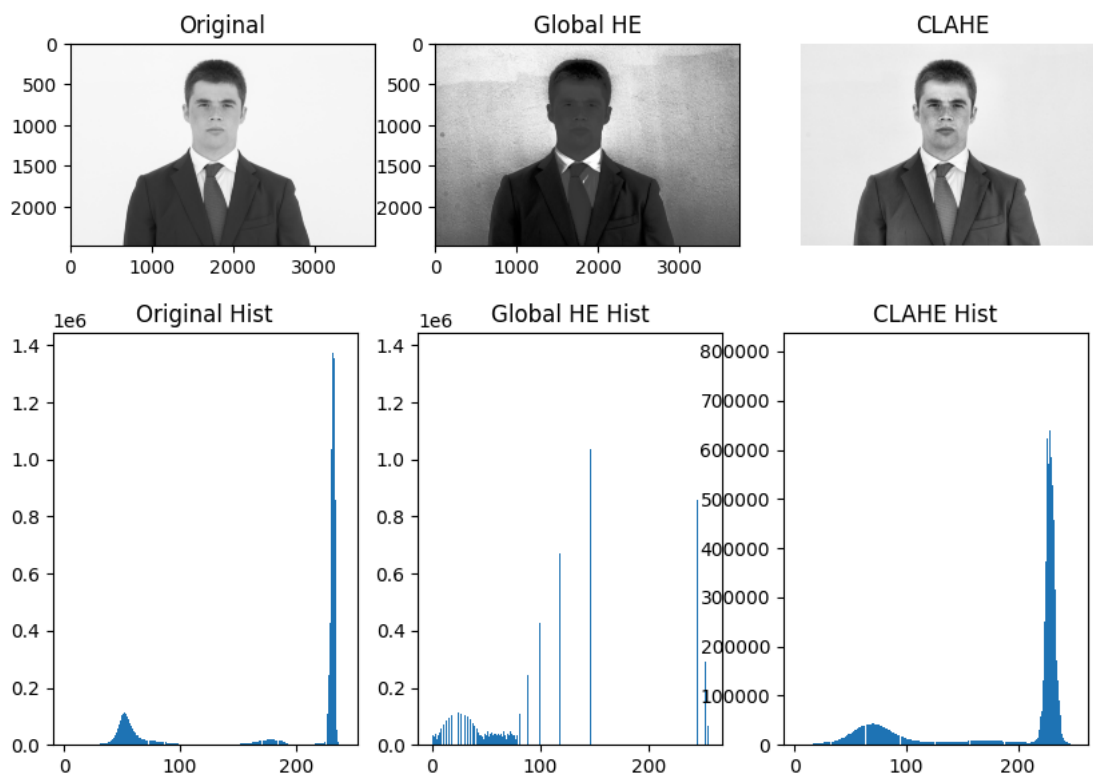
# Display histograms
plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.hist(gray.flatten(), bins=256)
plt.title("Original Hist")

plt.subplot(1,3,2)
plt.hist(hist_eq.flatten(), bins=256)
plt.title("Global HE Hist")

plt.subplot(1,3,3)
plt.hist(clahe_img.flatten(), bins=256)
plt.title("CLAHE Hist")

plt.show()
```



```
# 3. HISTOGRAM MATCHING
from google.colab import files
from skimage.exposure import match_histograms

print("Upload reference image for matching:")
uploaded2 = files.upload()

ref_img = cv2.imread(list(uploaded2.keys())[0])
ref_gray = cv2.cvtColor(ref_img, cv2.COLOR_BGR2GRAY)
```

```
# Match histogram
matched = match_histograms(gray, ref_gray)

# Display
plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Source")

plt.subplot(1,3,2)
plt.imshow(ref_gray, cmap='gray')
plt.title("Reference")

plt.subplot(1,3,3)
plt.imshow(matched, cmap='gray')
plt.title("Matched")

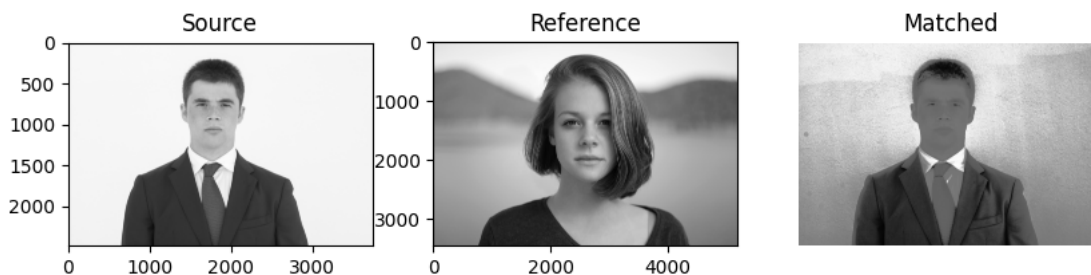
plt.axis('off')
plt.show()
```

Upload reference image for matching:

sample2.jpg

sample2.jpg(image/jpeg) - 1252174 bytes, last modified: 4/13/2026 - 100% done

Saving sample2.jpg to sample2.jpg



4. LOCAL ENHANCEMENT (COLOR)

```
# Convert RGB to LAB
lab = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2LAB)

# Split channels
L, A, B = cv2.split(lab)

# Apply CLAHE only on L (brightness)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
L_c1 = clahe.apply(L)

# Merge channels back
lab_c1 = cv2.merge((L_c1, A, B))

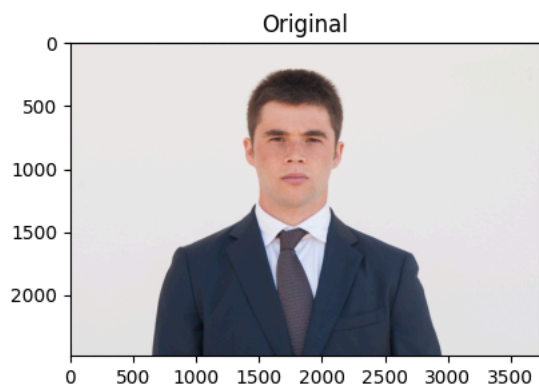
# Convert back to RGB
final_img = cv2.cvtColor(lab_c1, cv2.COLOR_LAB2RGB)

# Display
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original")

plt.subplot(1,2,2)
plt.imshow(final_img)
plt.title("CLAHE (L channel)")

plt.axis('off')
plt.show()
```



LAB 3

```
# =====
# Lab 3
# =====

# 1. SMOOTHING FILTERS

# Average Filter (Box Filter)
avg_3 = cv2.blur(gray, (3,3))
avg_7 = cv2.blur(gray, (7,7))

# Gaussian Filter
gauss_1 = cv2.GaussianBlur(gray, (5,5), 1.0)
gauss_2 = cv2.GaussianBlur(gray, (5,5), 2.0)

# Median Filter
med_3 = cv2.medianBlur(gray, 3)
med_7 = cv2.medianBlur(gray, 7)

# Display results
plt.figure(figsize=(12,8))

plt.subplot(2,3,1)
plt.imshow(avg_3, cmap='gray')
plt.title("Average 3x3")

plt.subplot(2,3,2)
plt.imshow(avg_7, cmap='gray')
plt.title("Average 7x7")

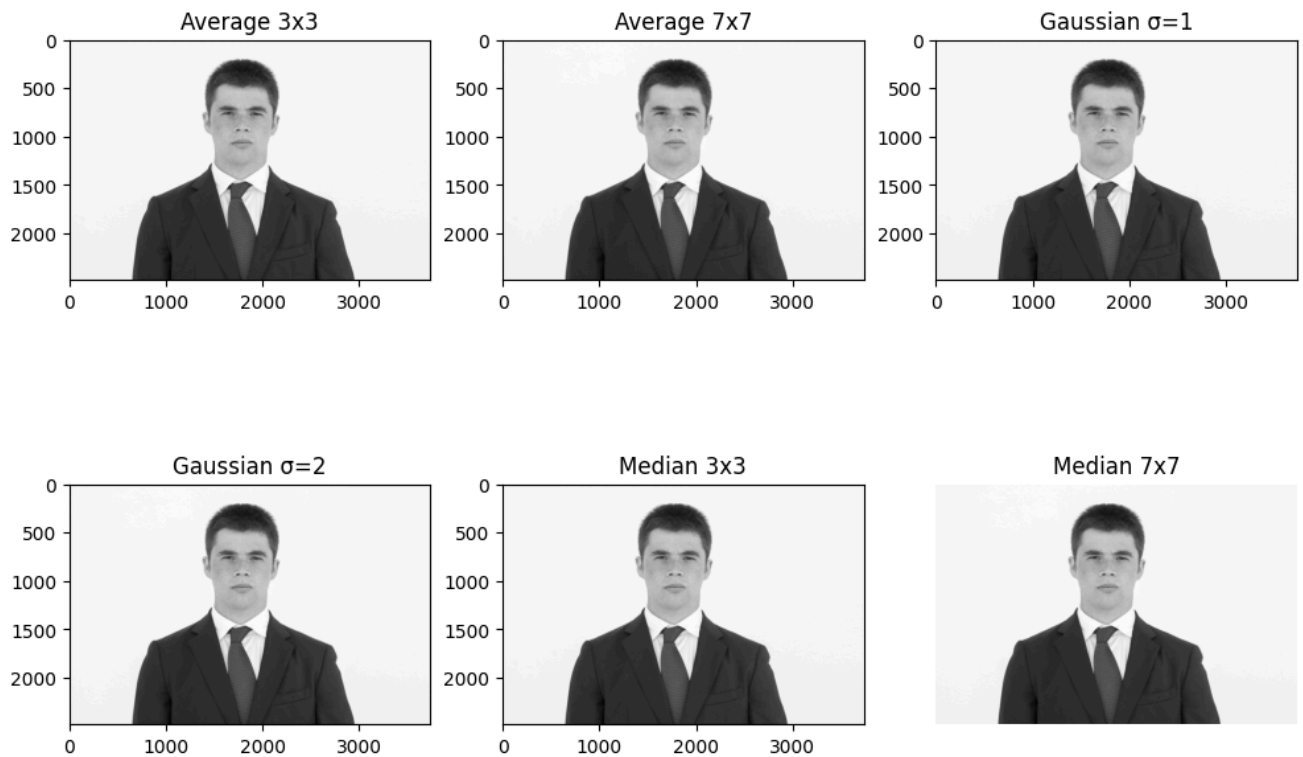
plt.subplot(2,3,3)
plt.imshow(gauss_1, cmap='gray')
plt.title("Gaussian σ=1")

plt.subplot(2,3,4)
plt.imshow(gauss_2, cmap='gray')
plt.title("Gaussian σ=2")

plt.subplot(2,3,5)
plt.imshow(med_3, cmap='gray')
plt.title("Median 3x3")

plt.subplot(2,3,6)
plt.imshow(med_7, cmap='gray')
plt.title("Median 7x7")

plt.axis('off')
plt.show()
```



2. SHARPENING FILTERS

```
# First blur image slightly
blur = cv2.GaussianBlur(gray, (5,5), 1)

# Laplacian Sharpening
laplacian = cv2.Laplacian(blur, cv2.CV_64F)
laplacian = np.uint8(np.absolute(laplacian))

sharp_lap = cv2.subtract(blur, laplacian)

# Unsharp Masking
gaussian = cv2.GaussianBlur(gray, (5,5), 1)
unsharp = cv2.addWeighted(gray, 1.5, gaussian, -0.5, 0)

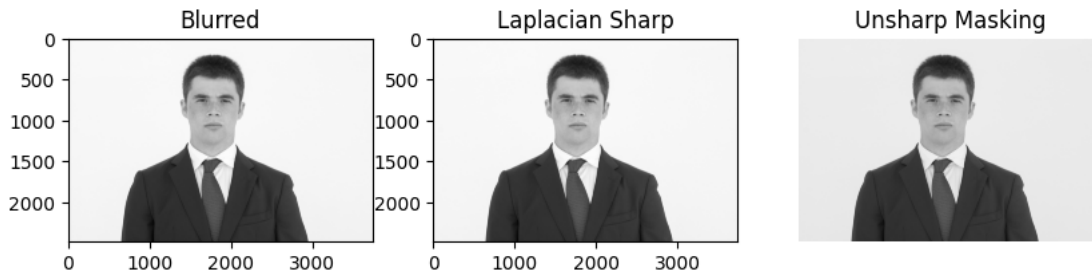
# Display
plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.imshow(blur, cmap='gray')
plt.title("Blurred")

plt.subplot(1,3,2)
plt.imshow(sharp_lap, cmap='gray')
plt.title("Laplacian Sharp")

plt.subplot(1,3,3)
plt.imshow(unsharp, cmap='gray')
plt.title("Unsharp Masking")

plt.axis('off')
plt.show()
```

3. ADD NOISE

```
# Gaussian Noise
mean = 0
sigma = 25
gaussian_noise = np.random.normal(mean, sigma, gray.shape)
noisy_gauss = gray + gaussian_noise
noisy_gauss = np.clip(noisy_gauss, 0, 255).astype(np.uint8)

# Salt & Pepper Noise
noisy_sp = gray.copy()
prob = 0.02

# Salt (white)
num_salt = int(prob * gray.size / 2)
coords = [np.random.randint(0, i-1, num_salt) for i in gray.shape]
noisy_sp[coords[0], coords[1]] = 255

# Pepper (black)
num_pepper = int(prob * gray.size / 2)
coords = [np.random.randint(0, i-1, num_pepper) for i in gray.shape]
noisy_sp[coords[0], coords[1]] = 0

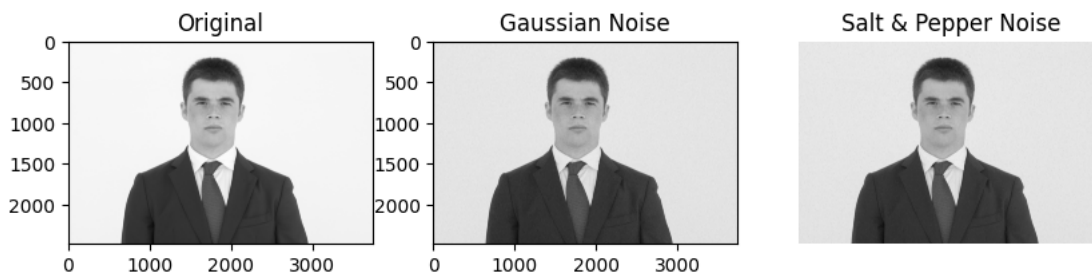
# Display
plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(noisy_gauss, cmap='gray')
plt.title("Gaussian Noise")

plt.subplot(1,3,3)
plt.imshow(noisy_sp, cmap='gray')
plt.title("Salt & Pepper Noise")

plt.axis('off')
plt.show()
```



4. FILTERING + PSNR

```
# Apply filters on noisy images

# Gaussian noise → Gaussian filter
gauss_filtered = cv2.GaussianBlur(noisy_gauss, (5,5), 1)

# Salt & Pepper → Median filter
sp_filtered = cv2.medianBlur(noisy_sp, 5)
```

```

# PSNR function
def compute_psnr(original, processed):
    mse = np.mean((original - processed) ** 2)
    if mse == 0:
        return 100
    max_pixel = 255.0
    psnr = 10 * np.log10((max_pixel ** 2) / mse)
    return psnr

# Compute PSNR
psnr_gauss = compute_psnr(gray, gauss_filtered)
psnr_sp = compute_psnr(gray, sp_filtered)

print("PSNR (Gaussian Noise → Gaussian Filter):", psnr_gauss)
print("PSNR (Salt & Pepper → Median Filter):", psnr_sp)

# Display
plt.figure(figsize=(10,4))

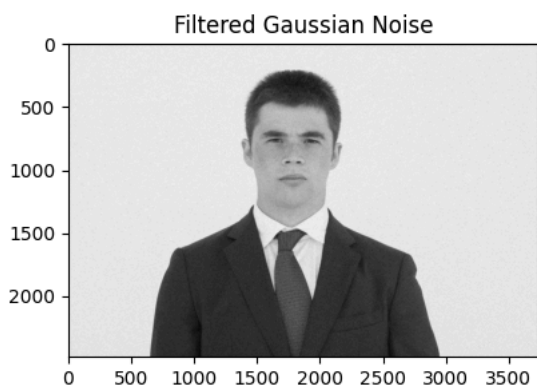
plt.subplot(1,2,1)
plt.imshow(gauss_filtered, cmap='gray')
plt.title("Filtered Gaussian Noise")

plt.subplot(1,2,2)
plt.imshow(sp_filtered, cmap='gray')
plt.title("Filtered Salt & Pepper")

plt.axis('off')
plt.show()

```

PSNR (Gaussian Noise → Gaussian Filter): 31.89369062710199
 PSNR (Salt & Pepper → Median Filter): 40.922784924308445



LAB 4

```

# =====
# Lab 4
# =====

# 1. 2D FFT (Magnitude & Phase)

# Compute FFT
f = np.fft.fft2(gray)

# Shift zero frequency to center
fshift = np.fft.fftshift(f)

# Magnitude spectrum (log scale for visibility)
magnitude = 20 * np.log(np.abs(fshift) + 1)

# Phase spectrum
phase = np.angle(fshift)

# Display
plt.figure(figsize=(10,4))

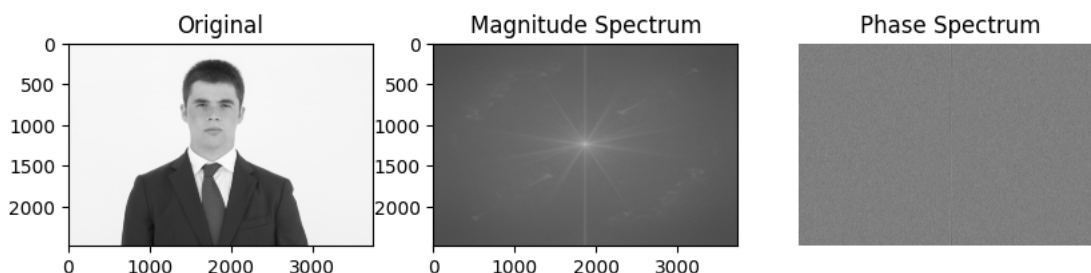
plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")

```

```
plt.subplot(1,3,2)
plt.imshow(magnitude, cmap='gray')
plt.title("Magnitude Spectrum")

plt.subplot(1,3,3)
plt.imshow(phase, cmap='gray')
plt.title("Phase Spectrum")

plt.axis('off')
plt.show()
```



2. FREQUENCY FILTERS

```
rows, cols = gray.shape
crow, ccol = rows//2 , cols//2

# Create distance matrix
x = np.arange(cols)
y = np.arange(rows)
X, Y = np.meshgrid(x, y)
D = np.sqrt((X - ccol)**2 + (Y - crow)**2)

# Ideal Low Pass Filter
D0 = 50
ideal_lpf = np.zeros((rows, cols))
ideal_lpf[D <= D0] = 1

# Gaussian Low Pass
gauss_lpf = np.exp(-(D**2) / (2*(D0**2)))

# Gaussian High Pass
gauss_hpf = 1 - gauss_lpf

# Apply filters
f_ideal = fshift * ideal_lpf
f_gauss = fshift * gauss_lpf
f_high = fshift * gauss_hpf

# Inverse FFT
img_ideal = np.abs(np.fft.ifft2(np.fft.ifftshift(f_ideal)))
img_gauss = np.abs(np.fft.ifft2(np.fft.ifftshift(f_gauss)))
img_high = np.abs(np.fft.ifft2(np.fft.ifftshift(f_high)))

# Display
plt.figure(figsize=(10,6))

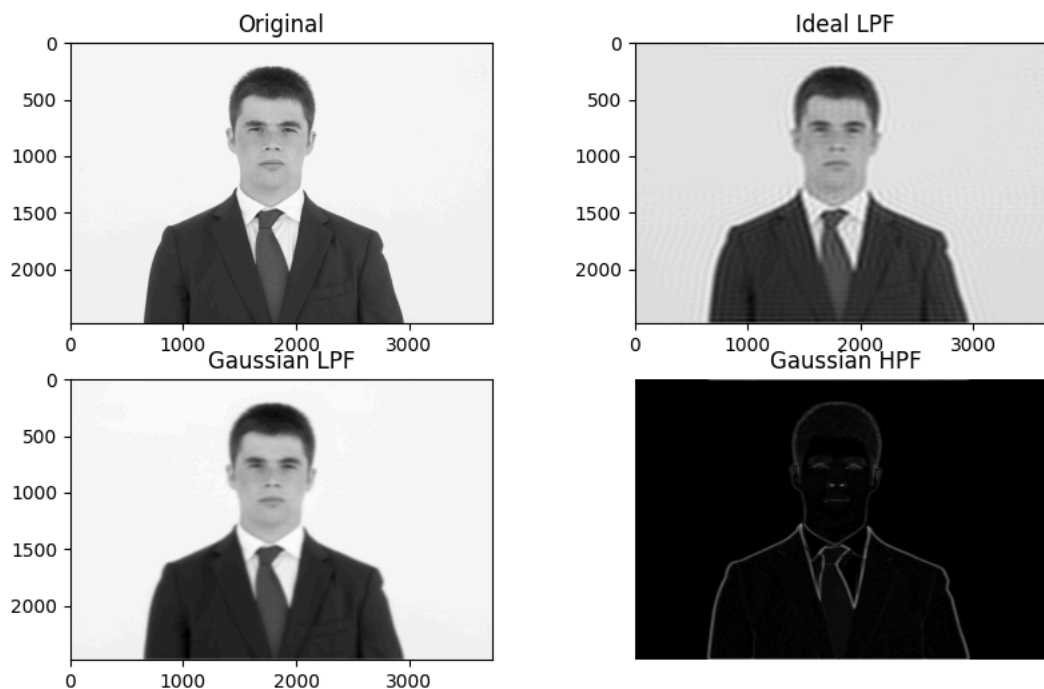
plt.subplot(2,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")

plt.subplot(2,2,2)
plt.imshow(img_ideal, cmap='gray')
plt.title("Ideal LPF")

plt.subplot(2,2,3)
plt.imshow(img_gauss, cmap='gray')
plt.title("Gaussian LPF")

plt.subplot(2,2,4)
plt.imshow(img_high, cmap='gray')
plt.title("Gaussian HPF")

plt.axis('off')
plt.show()
```



3. DOMAIN COMPARISON

```
# Frequency domain Gaussian (already computed above)
freq_blur = img_gauss

# Spatial domain Gaussian blur
spatial_blur = cv2.GaussianBlur(gray, (5,5), 1)

# PSNR function (reuse from Lab 3)
def compute_psnr(original, processed):
    mse = np.mean((original - processed) ** 2)
    if mse == 0:
        return 100
    return 10 * np.log10((255**2) / mse)

# Compare
psnr_val = compute_psnr(freq_blur.astype(np.uint8), spatial_blur)

print("PSNR between Frequency & Spatial Gaussian:", psnr_val)

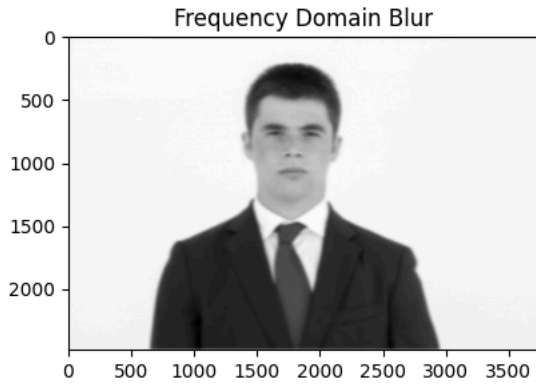
# Display
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(freq_blur, cmap='gray')
plt.title("Frequency Domain Blur")

plt.subplot(1,2,2)
plt.imshow(spatial_blur, cmap='gray')
plt.title("Spatial Domain Blur")

plt.axis('off')
plt.show()
```

PSNR between Frequency & Spatial Gaussian: 38.59071748571905



LAB 5

```
# =====
# Lab 5
# =====

# BASE
# FFT of image (No Chnage here)
f = np.fft.fft2(gray)
fshift = np.fft.fftshift(f)

rows, cols = gray.shape
crow, ccol = rows//2, cols//2

# Distance matrix
x, y = np.meshgrid(np.arange(cols), np.arange(rows))
D = np.sqrt((x - ccol)**2 + (y - crow)**2)

# -----

# 1. KERNEL FREQUENCY RESPONSE
def kernel_fft(kernel, shape):
    padded = np.zeros(shape)
    kh, kw = kernel.shape
    padded[:kh, :kw] = kernel

    f = np.fft.fft2(padded)
    fshift_kernel = np.fft.fftshift(f)

    # Better visualization
    magnitude = np.log(np.abs(fshift_kernel) + 1)
    magnitude = magnitude / np.max(magnitude)

    return magnitude

# Kernels
box = np.ones((3,3)) / 9

laplacian = np.array([[0,-1,0],
                      [-1,4,-1],
                      [0,-1,0]])

sobel = np.array([[ -1,0,1],
                  [-2,0,2],
                  [-1,0,1]])

# Compute FFT
box_f = kernel_fft(box, gray.shape)
lap_f = kernel_fft(laplacian, gray.shape)
sobel_f = kernel_fft(sobel, gray.shape)

# Display
plt.figure(figsize=(12,4))

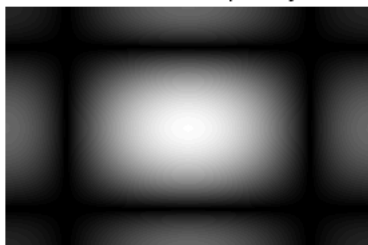
plt.subplot(1,3,1)
plt.imshow(box_f, cmap='gray')
plt.title("Box Filter Frequency")
plt.axis('off')
```

```
plt.subplot(1,3,2)
plt.imshow(lap_f, cmap='gray')
plt.title("Laplacian Frequency")
plt.axis('off')

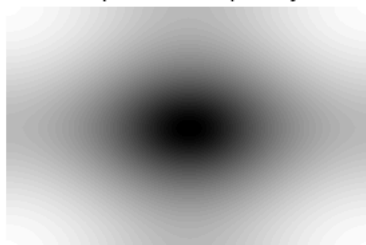
plt.subplot(1,3,3)
plt.imshow(sobel_f, cmap='gray')
plt.title("Sobel Frequency")
plt.axis('off')

plt.show()
```

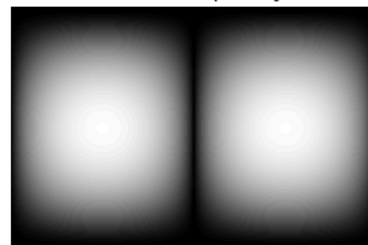
Box Filter Frequency



Laplacian Frequency



Sobel Frequency



```
# Gaussian LPFs
low_small = np.exp(-(D**2)/(2*(10**2)))
low_large = np.exp(-(D**2)/(2*(50**2)))

# Band-pass
band_pass = low_large - low_small

# Apply filter (IMPORTANT: use image fshift)
f_band = fshift * band_pass

# Inverse FFT
band_img = np.abs(np.fft.ifft2(np.fft.ifftshift(f_band)))

# Normalize (FIX BLACK IMAGE)
band_img = cv2.normalize(band_img, None, 0, 255, cv2.NORM_MINMAX)

# Display
plt.imshow(band_img, cmap='gray')
plt.title("Band-Pass Filter Result")
plt.axis('off')
plt.show()
```

Band-Pass Filter Result



```
# Gaussian LPF
gauss_lpf = np.exp(-(D**2)/(2*(30**2)))

# Parameters
k1 = 0.5
k2 = 1.5
```

```
# High Frequency Emphasis
hfe = k1 + k2 * (1 - gauss_lpf)

# Apply
f_hfe = fshift * hfe
img_hfe = np.abs(np.fft.ifft2(np.fft.ifftshift(f_hfe)))

# Normalize (FIX BLACK IMAGE)
img_hfe = cv2.normalize(img_hfe, None, 0, 255, cv2.NORM_MINMAX)

# Display
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(img_hfe, cmap='gray')
plt.title("High Frequency Emphasis")
```